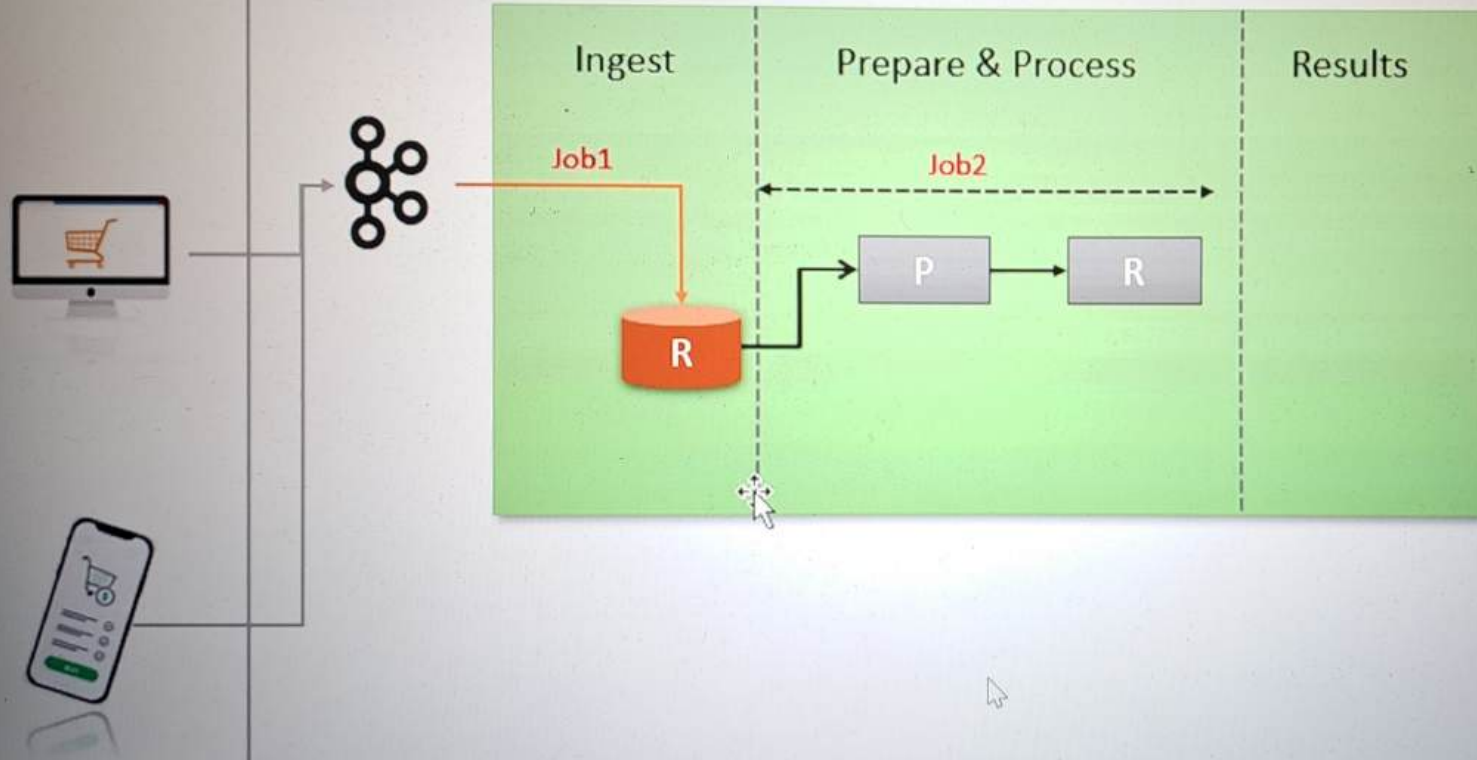


Source System

Data Engineering Platform



Cmd 1

Python

```
class Bronze():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"
        self.Bootstrap_SERVER = "pkc-6ojv2.us-west4.gcp.confluent.cloud:9092"
        self.JAAS_MODULE = "org.apache.kafka.common.security.plain.PlainLoginModule"
        self.CLUSTER_API_KEY = "TQ7F3730G6J4TT4G"
        self.CLUSTER_API_SECRET = "ftb0mW0YcCT6oAG9GSIjyUxHYuJmbUPDi0G0c5ymt0t6FKJ1J1IVDbDRVqKoG5H4"

    def ingestFromKafka(self, startingTime = 1):
        return ( spark.readStream
                .format("kafka")
                .option("kafka.bootstrap.servers", Bootstrap_SERVER)
                .option("kafka.security.protocol", "SASL_SSL")
                .option("kafka.sasl.mechanism", "PLAIN")
                .option("kafka.sasl.jaas.config", f"{JAAS_MODULE} required username='{CLUSTER_API_KEY}' password='{CLUSTER_API_SECRET}';")
                .option("maxOffsetsPerTrigger", 10)
                .option("subscribe", "invoices")
                .option("startingTimestamp", startingTime)
                .load()
        )

    def getSchema(self):
        return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,
        CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,
        PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
```



```
.option("kafka.bootstrap.servers", BOOTSTRAP_SERVER)
.option("kafka.security.protocol", "SASL_SSL")
.option("kafka.sasl.mechanism", "PLAIN")
.option("kafka.sasl.jaas.config", f"{JAAS_MODULE} required username='{CLUSTER_API_KEY}' password='{CLUSTER_API_SECRET}';")
.option("maxOffsetsPerTrigger", 10)
.option("subscribe", "invoices")
.option("startingTimestamp", startingTime)
.load()
```

```
def getSchema(self):
```

```
    return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,
    CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,
    PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
    DeliveryType string,
    DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,
    State string>,
    InvoiceLineItems array<struct<ItemCode string, ItemDescription string,
    ItemPrice double, ItemQty bigint, TotalValue double>>
```

```
def getInvoices(self, kafka_df):
```

```
    from pyspark.sql.functions import from_json
    return ( kafka_df.select(kafka_df.key.cast("string").alias("key"),
        from_json(kafka_df.value.cast("string"), self.getSchema()).alias("value"),
        "topic", "timestamp")
    )
```




```
DeliveryType string,  
DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,  
State string>,  
InvoiceLineItems array<struct<ItemCode string, ItemDescription string,  
ItemPrice double, ItemQty bigint, TotalValue double>>  
.....
```

```
def getInvoices(self, kafka_df):  
    from pyspark.sql.functions import from_json  
    return ( kafka_df.select(kafka_df.key.cast("string").alias("key"),  
                             from_json(kafka_df.value.cast("string"), self.getSchema()).alias("value"),  
                             "topic", "timestamp")  
    )  
  
def process(self, startingTime = 1):  
    print(f"Starting Bronze Stream...", end='')  
    kafka_df = self.ingestFromKafka(startingTime)  
    invoices_df = self.getInvoices(kafka_df)  
    sQuery = ( invoices_df.writeStream  
               .queryName("bronze-ingestion")  
               .option("checkpointLocation", f"{self.base_data_dir}/checkpoint/invoices_bz")  
               .outputMode("append")  
               .toTable("invoices_bz")  
    )  
    print("Done")  
    return sQuery
```



11-kafka-to-bronze-test-suite Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all

Connect

Share

Publish

%run ./10-kafka-to-bronze

Cmd 2

Python

```
class kafkaToBronzeTestSuite():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def cleanTests(self):
        print(f"Starting Cleanup...", end='')
        spark.sql("drop table if exists invoices_bz")
        dbutils.fs.rm("/user/hive/warehouse/invoices_bz", True)
        dbutils.fs.rm(f"{self.base_data_dir}/checkpoint/invoices_bz", True)
        print("Done")

    def assertResult(self, expected_count):
        print(f"\tStarting validation...", end='')
        actual_count = spark.sql("select count(*) from invoices_bz").collect()[0][0]
        assert expected_count == actual_count, f"Test failed! actual count is {actual_count}"
        print("Done")

    def waitMicroBatch(self, sleep=30):
        import time
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
```



```
def runtests(self):
    self.cleanTests()
    bzStream = Bronze()

    print("Testing Scenarion - Start from beginneing on a new checkpoint...")
    bzQuery = bzStream.process()
    self.waitForMicroBatch()
    bzQuery.stop()
    self.assertResult(30)
    print("Validation passed.\n")

    print("Testing Scenarion - Restart from where it stopped on the same checkpoint...")
    bzQuery = bzStream.process()
    self.waitForMicroBatch()
    bzQuery.stop()
    self.assertResult(30)
    print("Validation passed.\n")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text




```
self.assertResult(30)
print("Validation passed.\n")

print("Testing Scenarion - Restart from where it stopped on the same checkpoint...")
bzQuery = bzStream.process()
self.waitForMicroBatch()
bzQuery.stop()
self.assertResult(30)
print("Validation passed.\n")

print("Testing Scenario - Start from 1697945539000 on a new checkpoint...")
dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/invoices_bz", True)
bzQuery = bzStream.process(1697945539000)
self.waitForMicroBatch()
bzQuery.stop()
self.assertResult(40)
print("Validation passed.\n")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



11-kafka-to-bronze-test-suite Python ▾

File Edit View Run Help [Last edit was now](#) [Provide feedback](#)

▶ Run all

● Connect ▾

Share

Publish

```
bzQuery = bzStream.process(1697945539000)
self.waitForMicroBatch()
bzQuery.stop()
self.assertResult(40)
print("Validation passed.\n")
```

Cmd 3

Python



```
ts = kafkaToBronzeTestSuite()
ts.runTests()
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

